

Second Module

Marco Barberio

Master Degree in Computer Science

m.barberio2@studenti.unipi.it

Parallel and Distributed Systems: Paradigms and Models, Academic Year: 2025/2026

1 Parallel Implementation

Starting from the sequential reference implementation, I developed a parallel version of the partitioned hash join while preserving the overall structure of the algorithm. My implementation is based on a thread pool, which is reused across the different stages of the computation. This choice avoids the overhead of repeatedly creating and destroying threads, which would be particularly undesirable in an algorithm organized as a sequence of relatively short phases.

The first part of the pipeline is the partitioning of the two input relations. As in the sequential version, each relation is processed through three main steps: histogram computation, exclusive prefix sum, and scatter into a partitioned output array. Among these phases, the histogram is naturally well suited for parallelism. Each worker thread processes a disjoint chunk of the input relation and computes a private local histogram, counting how many records fall into each partition. Once all threads complete their local work, the local histograms are reduced into the final global histogram. This approach avoids contention on shared counters and keeps synchronization costs low, since communication between threads is limited to the final reduction step.

The scatter phase was also parallelized, but with a different strategy. A naive parallel scatter based on shared write pointers would require synchronization on every insertion, introducing contention and reducing scalability. To avoid this issue, I exploited the information already produced during the histogram phase. For each partition, I computed the starting write offset associated with each thread, so that every worker owns a private region of the output array for that partition. During scatter, each thread writes only inside its own precomputed ranges and therefore no atomic operations or locks are needed in the inner loop. This makes the phase both thread-safe and efficient, while preserving the same final layout as the sequential implementation.

After partitioning, the join phase is the next phase that I parallelized. Once the two relations have been rearranged so that records belonging to the same partition are stored contiguously, each

partition can be processed independently from the others. This makes partition-level parallelism a natural choice. In my implementation, worker threads repeatedly fetch the next partition identifier to process and execute the local join on that partition. For each partition, the corresponding segment of relation R is first used to build a local lookup structure containing key multiplicities, and the corresponding segment of relation S is then probed against it. Since duplicates are allowed, each match contributes according to the multiplicity found in the build side. Each thread accumulates its own local `join_count`, `checksum1`, and `checksum2`, which are merged only at the end.

A dynamic assignment of partitions to threads was preferred over a static one. In the implementation, this is achieved through a shared atomic counter from which each worker fetches the identifier of the next partition to process as soon as it completes the current one. Thus, partitions are not assigned to threads in advance, but are claimed on demand during execution. This choice improves load balancing, since the processing cost of a partition may vary even when the mapping function distributes keys reasonably well. In fact, the join cost depends not only on partition cardinality, but also on the number of duplicates, on the number of successful matches generated during probing, and on the cost of building and querying the local hash-based structure. Dynamic scheduling therefore reduces the risk that some threads become idle while others are still processing more expensive partitions. Overall, the adopted strategy focuses on the phases with the highest amount of independent work and the lowest synchronization cost. Histogram computation, scatter, and partition-wise join were parallelized because they expose substantial concurrency and dominate the execution time of the pipeline. In contrast, the prefix sum and the final reduction were left sequential because their contribution is limited and their parallelization would not significantly improve end-to-end performance. This design keeps the implementation close to the original algorithm while making the main computational bottlenecks effectively parallel.

2 Experimental Results

2.1 Experimental Setup

All experiments were conducted on node-08 of the `spmcluster`, with the number of partitions fixed to $P = 1024$. In all reported measurements, the thread count denotes the number of worker threads in the thread pool. Execution times were collected end to end over the parallel join pipeline and thus also account for the orchestration and synchronization work carried out by the main thread

within the measured region. Results were averaged over multiple runs using different random seeds. Both strong and weak scaling experiments were performed to assess the behavior of the parallel implementation under different workloads.

2.2 Strong Scaling

Figure 1 shows the speedup obtained by increasing the number of threads while keeping the input size fixed ($N = 10^7$). The observed speedup increases with the number of threads, reaching approximately $10\times$ at 32 threads. However, it remains below the ideal linear speedup.

This behavior can be explained by the presence of memory-bound phases, such as scatter, and by synchronization and scheduling overheads that become more significant at higher levels of parallelism. Additionally, contention on shared resources, such as memory bandwidth, limits scalability.

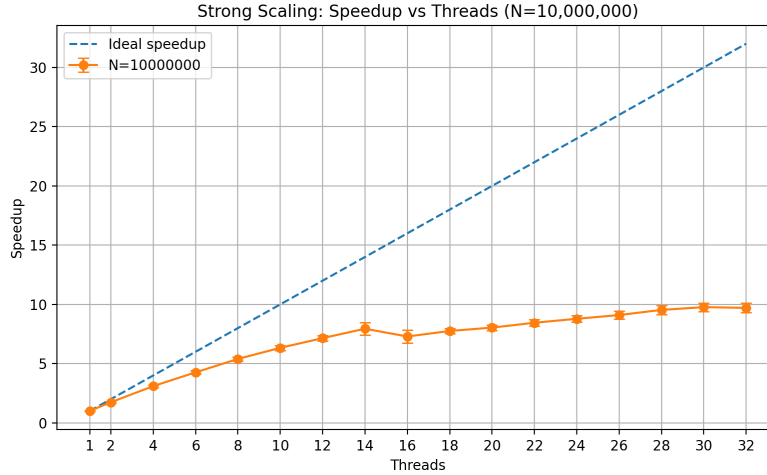


Figure 1: Strong scaling: speedup vs number of threads.

2.3 Weak Scaling

In the weak scaling experiment, the problem size increases proportionally to the number of threads. Ideally, the execution time should remain constant. Figure 2 reports the weak scaling efficiency. The efficiency decreases as the number of threads increases, dropping to around 40% at 32 threads. This indicates that the system becomes increasingly limited by memory and coordination overheads.

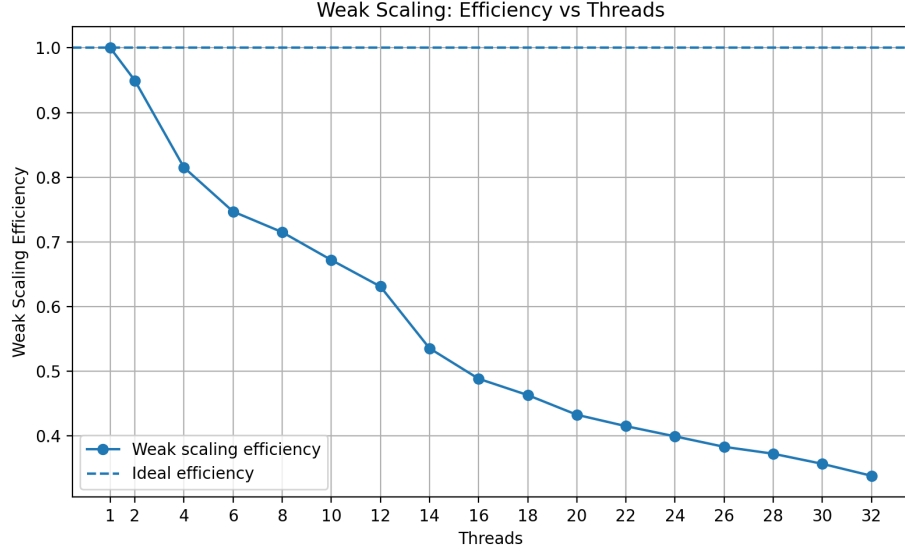


Figure 2: Weak scaling: efficiency vs number of threads.

2.4 Phase Breakdown

Figure 3 reports the execution time split across the main phases of the algorithm.

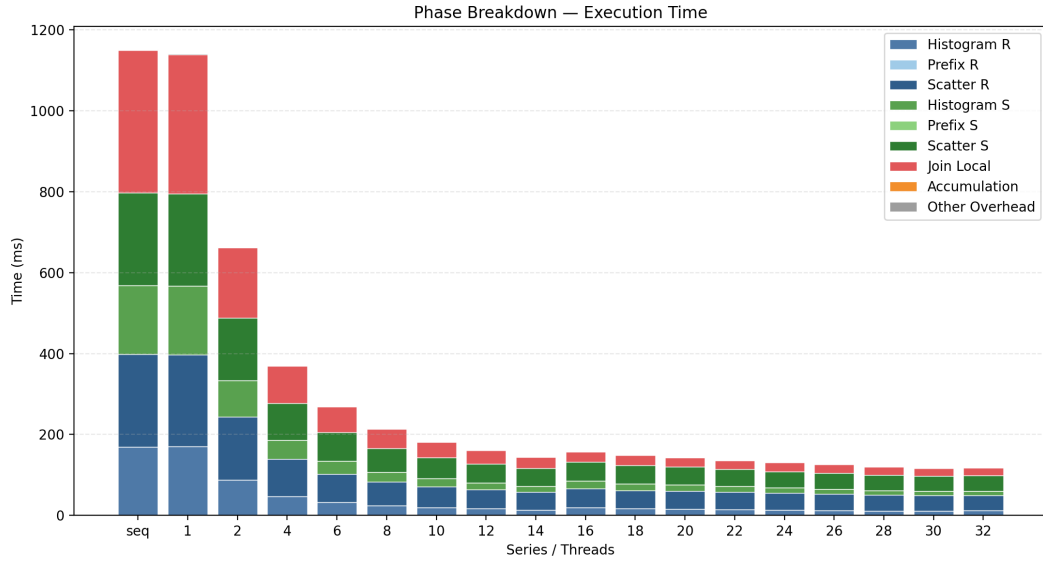


Figure 3: Execution time breakdown across phases.

The figure shows that histogram computation and partition-wise join benefit the most from parallelization. In both cases, the work can be distributed across threads with limited synchronization, leading to a clear reduction in execution time as the number of threads increases. The join phase, in particular, also benefits from dynamic scheduling, which improves load balancing

when different partitions have different processing costs.

The scatter phase shows a more limited improvement. Although it is parallelized without synchronization in the inner loop, each thread still performs irregular writes to memory, making this phase more memory-bound and therefore less scalable than histogram computation and join.

2.5 Discussion

Overall, the experimental results show that the adopted parallelization strategy is effective. The phases that dominate the execution time and expose the largest amount of independent work, namely histogram computation, scatter, and partition-wise join, were parallelized while preserving the structure of the original algorithm. In particular, per-thread local histograms reduce contention during partitioning, thread-private output ranges remove synchronization from the inner loop of scatter, and dynamic scheduling improves load balancing during the join phase.

At the same time, the results show that the implementation does not achieve ideal scaling at higher thread counts. This behavior is not mainly due to the few phases left sequential, since prefix sum and final accumulation contribute only marginally to the total execution time. Rather, the main limitation comes from memory behavior, especially in phases such as scatter, together with the increasing coordination overheads that arise at higher levels of parallelism.

In summary, the implementation achieves a reasonable trade-off between simplicity and performance: the most relevant phases of the pipeline were parallelized, while the phases left sequential were those for which additional complexity would not provide end-to-end benefit.